

1. (1 punto) Complete indicando qué imprimen los siguientes códigos:

a)

```
int i = 6
do
    printf("%d", --i);
while (--i);
```

b)

```
int i = 1 ;
switch (i) {
    case 'a': printf("A");
    case 2: printf("B");
    default: printf("Z");
}
```

c)

```
int i = 5, j = 2;
while (i -- && j++)
    printf("%d", i);
```

d)

```
int i = 0;
for (i = 32; i > 0; i >>=1)
    printf("%d", i);
```

2. (1.3 puntos) Complete indicando que imprime el siguiente código:

```
int acum (int n) {
    static int total = 0;
    total += n;
    return total;
}
int main () {
    int i;
    for (i = 1; i < 4; i++)
        printf("i = %d\n", acum (i));
    return 0;
}
```

3. (1.8 puntos) Marque con verdadero (no genera error) o falso (genera error)

int *p; int x; int a[10]

- | | | |
|------------------------------------|--------------------------------|--|
| a) <code>p = &a[10];</code> | e) <code>x = *(a+2);</code> | i) <code>*p = *(&x);</code> |
| b) <code>x = *(p+5);</code> | f) <code>&x = p;</code> | j) <code>a = p+1;</code> |
| c) <code>p = &(&x);</code> | g) <code>p = a++ ;</code> | k) <code>p = a+1;</code> |
| d) <code>x = *p;</code> | h) <code>a[1] = *(p+0);</code> | l) <code>x= &a[0] - &a[5]</code> |

4. (1.8 punto) Para cada inciso indique si es verdadero o falso

- a) Se puede inicializar una unión con más de un campo a la vez.
- b) `sizeof(únión)` es igual al `sizeof` del campo más grande.
- c) `typedef` crea un nuevo tipo,
- d) Un campo de bits no puede leerse mediante la función `scanf()`.
- e) Un campo de bits no puede imprimirse mediante la función `printf()`.
- f) Es lo mismo declarar una variable local como `int` que como `auto int`.
- g) Los arreglos se pueden copiar con una asignación siempre que tengan el mismo tipo.
- h) Una variable **register** garantiza que se almacene en un registro del procesador.
- i) El tamaño de un **struct** puede ser mayor que la suma de sus campos.



5. (1,3 puntos) Implemente la función **trim left** reciba una cadena **in** y retorne en la cadena **out** una versión de la cadena **in** sin espacios iniciales.

ejemplo: in: "00Hola mundo!" - out: "Hola Mundo!"

```
void trim_left(const char in[] ; char out [] ) {
```

6. (2.8 puntos) Se requiere implementar un módulo para registrar los resultados de una mesa de examen. De cada alumno se almacena su legajo y su nota. El módulo debe manejar un código de materia (entero) y notas enteras de 0 a 10, usando -1 para indicar ausencia.

a. Definición de tipos:

1. Defina un tipo `legajo_t` eficiente, en cuanto a memoria, para representar un legajo. El mismo está compuesto por el número de alumno (al menos 5 dígitos) y un dígito verificador.
2. Defina el tipo `nota_examen_t` que agrupe el legajo y la nota del estudiante (valor entre -1 y 10).
3. Defina el tipo enumerativo `stats_mesa_t` con los valores Ausente, Desaprobado y Aprobado.

b. Definición de funciones:

1. Defina la función `stats_mesa_str` que reciba un valor de tipo `stats_mesa_t` y **retorne** su representación como cadena de texto.
2. Defina la función `cargar_nota_examen` que cargue los datos de un examen a partir de valores ingresados por teclado y retorne un valor de tipo `nota_examen_t`.
3. Defina la función `estadistica_mesa` que reciba un arreglo de `nota_examen_t` y su cantidad, y **retorne** un arreglo con el conteo de Ausentes (nota =- 1), Desaprobados (nota de 0 a 4) y Aprobados (nota de 5 a 10).

c. Implemente un main que: cargue los datos de 25 estudiantes, calcule las estadísticas de la mesa e imprima cada resultado junto a su nombre descriptivo.

